Functional Versus Imperative Is Solved

Power, Cost, and Shared State in the Paradigm Debate

29 October 2026

1 Two Ways to Sum a List

Here is the same function written twice.

Listing 1: Imperative, in C.

```c
int total(const int *xs, int n) {
  int acc = 0;
  for (int i = 0; i < n; i++)
    acc += xs[i];
  return acc;
}
```

Listing 2: Functional, in Haskell.

```haskell
total :: [Int] -> Int
total []       = 0
total (x : xs) = x + total xs
```

The first is a list of instructions. The second is a pair of equations. Programmers have argued about which is better for most of the history of programming, and the argument is older than most of the languages it is now conducted in. We settle it.

Definition 1.1 (Imperative)**.** A program is *imperative* if its meaning is a sequence of commands, each of which changes a *store*, a map from locations to values. Running it executes the commands in order, and its result is read from the final store.

Definition 1.2 (Functional)**.** A program is *functional* if its meaning is an expression, and the meaning of an expression is its value. It is *pure* if the value of every expression depends only on the values of its subexpressions, so that any subexpression can be replaced by its value without changing the meaning of the whole. Strachey, borrowing from Quine, called this property *referential transparency* [20].

Remark 1.3. The term has since been used in several senses that are not equivalent [18]. We use Strachey's. Both definitions classify programs, not languages. C permits a function that assigns nothing, and Haskell permits one that does nothing but assign. Most languages in use permit both, so the question is not which language to use but which kind of program to write.

2 The Ceasefire of 1937

The first question, whether one style can compute something the other cannot, was settled before either had a machine to run on. Church proposed that the effectively calculable functions are those definable in the λ-calculus [3]. Turing proposed those computable by a machine that reads and writes symbols on a tape, following a table of instructions [22]. The first is a functional language and the second an imperative one.

Theorem 2.1 (The Church–Turing Ceasefire)**.** *A function on the natural numbers is λ-definable if and only if it is computable by a Turing machine.*

Proof. Turing showed that every λ-definable function is computable and every computable function is general recursive [23], and Kleene that every general recursive function is λ-definable [11]. □

Corollary 2.2 (No Exclusive Programs)**.** *Any function computed by a program in one Turing-complete language, functional or imperative, is computed by some program in any other.*

Proof. Each such language computes at least the functions that Turing machines compute, since it is Turing-complete, and at most these, since a Turing machine can run its interpreter. Theorem 2.1 is the case of the λ-calculus and Turing's machines. □

Remark 2.3. The ceasefire was signed in 1937, eleven years before the first stored-program computer ran a program [24]. It concerns which functions can be computed, not how quickly, how clearly, or how often incorrectly. The rest of this paper is about those.

3 Every Program Is Both

The ceasefire is not only a theorem. Compilers carry it out, in both directions, on most programs that are ever run.

Proposition 3.1 (Lambda, the Ultimate Goto)**.** *A call in tail position, whose result the caller returns unchanged, can be compiled as a jump that reuses the caller's stack frame. A tail-recursive function therefore runs as a loop, in constant stack space [19].*

A functional programmer who wants the loop of Listing 1 writes Listing 3, which GHC compiles to a loop.

Listing 3: Listing 1, as a functional programmer writes it.

```
total :: [Int] -> Int
total xs = go xs 0
  where
    go []       acc = acc
    go (y : ys) acc = go ys (acc + y)
```

Proposition 3.2 (SSA Is Functional Programming)**.** *In* static single assignment *(SSA) form every variable is assigned exactly once, and where control flow merges, a ϕ-function selects a value according to the block that control came from [4]. A program in SSA form can be read as a functional program: each block is a function, each ϕ-function a parameter, and each jump a tail call [10, 1].*

Listing 4: Listing 1 in SSA form.

```
entry:
  br loop
loop:
  i   = phi [0, entry], [i1, body]
  acc = phi [0, entry], [acc1, body]
  br i < n, body, exit
body:
  acc1 = acc + xs[i]
  i1   = i + 1
  br loop
exit:
  ret acc
```

Listing 5: Listing 4, read as functions.

```
entry()      = loop(0, 0)
loop(i, acc) = if i < n then body(i, acc)
               else exit(acc)
body(i, acc) = loop(i + 1, acc + xs[i])
exit(acc)    = acc
```

Clang compiles C to LLVM IR, which is in SSA form throughout [12], and the `mem2reg` pass promotes local variables whose addresses are never taken out of memory and into SSA form [13]. GCC has optimised in SSA form since version 4.0, in 2005 [7]. Arrays and other memory remain loads and stores, so Appel's reading covers the scalar variables [1], but the loop of Listing 1 spends most of its time in the compiler as the functions of Listing 5. C programmers have been writing functional programs for decades. The compiler was too polite to mention it.

The traffic runs the other way too. GHC translates Haskell through two functional intermediate languages, Core and STG, into Cmm, "a low-level imperative language with an explicit stack", and from there into machine code, directly or through LLVM [14]. Lazy evaluation, which in the words of Haskell's designers "kept us pure" [9], is itself implemented by mutation. An argument is passed unevaluated, as a *thunk*, and the first time its value is needed the thunk is overwritten in place with that value, so that it is computed at most once. Wadsworth proposed this in 1971 [26], and GHC's STG machine works this way [16].

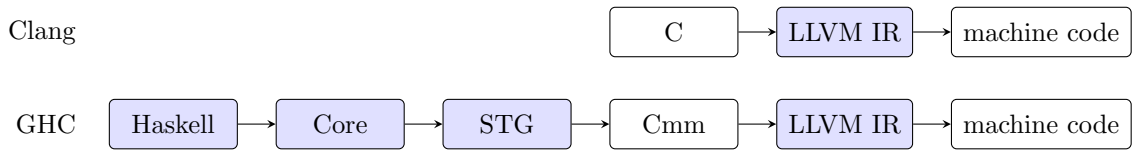


Figure 1: The stages of two compilers, with functional stages shaded: Clang, and GHC with its LLVM backend. Reading SSA form as functional (Proposition 3.2), the C program changes paradigm twice and the Haskell program three times.

Remark 3.3. The paradigm of a program is therefore a property of the stage at which a person reads it. Most of the debate concerns the first stage.

4 Equally Powerful, Not Equally Expressive

Corollary 2.2 allows a translation to rewrite the whole program, which is how compilers use it. Programmers would rather not. Felleisen asked instead which features can be removed by *macro* translation, rewriting each use of a feature from its parts without touching the rest of the program [6]. His test is observational: if a new feature lets some program tell apart two expressions that no program could tell apart before, no macro can remove it.

Theorem 4.1 (Felleisen)**.** *In Pure Scheme, a call-by-value functional language, the expressions* `((lambda (d) (f 0)) (f 0))` *and* `(f 0)`*, which call* `f` *twice and once, are operationally equivalent. Assignment, through* `set!` *and* `letrec`*, separates them, so Pure Scheme cannot macro-express assignment [6].*

Sketch. Without assignment, a function called twice on the same argument does the same thing both times, so the first call can matter only by never returning, and then the single call never returns either. With assignment, `f` can redefine itself on its first call so that its second call never returns. □

Felleisen proves the same for first-class continuations. The theorem's witness is a familiar maintenance problem. Suppose a function deep inside a pure program must now count how often it is called, which is exactly the difference between calling it twice and calling it once. In C this is one global variable and one increment. In Haskell the function's type changes, from `a -> b` to `a -> State Int b`, and so does the type of every function that calls it, directly or not. The imperative programmer pays the same price elsewhere. Keeping the previous version of a data structure, to undo an edit or to read it from another thread, is free in a pure program, where old versions are never destroyed, and needs a copy before every update, or a redesigned structure [5], in an imperative one. Table 1 collects these and two more.

Change	Functional	Imperative
Count the calls to a deep function	every caller	one variable
Keep the previous version of a structure	nothing	every update
Add an operation on a data type	one function	every class
Add a case to a data type	every function	one class

Table 1: What a change touches in each style. The last two rows assume the object-oriented style common in imperative languages, and are the Expression Problem.

Wadler named the last two rows the *Expression Problem*: to define a data type by cases, "where one can add new cases to the datatype and new functions over the datatype, without recompiling existing code, and while retaining static type safety" [25]. Solutions exist, and rely on type-system features developed for the purpose [21].

Remark 4.2 (Conservation of Inconvenience)*.* For changes that cross function boundaries, each style makes some local by making others global. Felleisen's measure ranks languages, not styles, and it rewards having everything: any feature that no macro can remove makes a language more expressive, which may explain C++.

5 The Price of Purity

A pure program cannot update memory in place. Where C overwrites one element of an array, a pure program builds a new array, or a new path through a tree that shares everything else with the old one. Pippenger asked what this costs, in a model of Lisp in which purity means giving up `rplaca` and `rplacd`, the operations that overwrite a cons cell [17]. His programs are *on-line*, producing each output before reading the next input, and *symbolic*, comparing input symbols only for equality.

Proposition 5.1 (The Logarithm Bound)**.** *An impure symbolic on-line program that performs T operations can be simulated by a pure one that performs $O(T \log T)$ [17].*

Proof. Keep the cells that the impure program overwrites in a balanced tree, keyed by when they were created, and copy the path to a cell when it is updated. With at most T cells the tree has depth $O(\log T)$, so each access costs $O(\log T)$, and each new version of the tree shares all but one path with the old. □

Theorem 5.2 (Pippenger)**.** *There is a symbolic on-line problem that an impure Lisp program solves with $O(n)$ operations, while every pure Lisp program needs $\Omega(n \log n)$ operations on some inputs [17].*

The logarithm is therefore sometimes charged, in a model that evaluates everything as soon as it can. Lazy evaluation changes the answer.

Theorem 5.3 (Bird, Jones and de Moor)**.** *With lazy evaluation, a pure program solves Pippenger's problem in linear time overall [2].*

Remark 5.4 (The Refund)*.* Laziness recovers the logarithm through the mechanism of Section 3: an evaluated thunk is overwritten with its value. Bird, Jones and de Moor call this "essentially a restricted memoization", and the capsule review printed with their paper calls it "this restricted form of mutation". They "do not, and cannot, claim that a lazy implementation can solve all problems with the same efficiency as an impure Lisp solution" [2], and as far as we know the question is still open. Purity costs at most a logarithm, and on the problem that shows the logarithm is ever charged, laziness refunds it.

Many structures pay nothing (Table 2). Hood and Melville gave a purely functional queue whose every operation takes constant time in the worst case [8]. Okasaki's simpler one relies on laziness, and he concedes that "some might argue that our implementation is not, in fact, purely functional because of the imperative nature of memoization" [15]. Persistence, the ability to read old versions after an update, comes free with purity, and costs an imperative linked structure of bounded in-degree $O(1)$ amortised time and space per update [5].

Operation	Imperative	Purely functional
Read or write an array element	$O(1)$	$O(\log n)$
Add to or remove from a queue	$O(1)$	$O(1)$ [8]
Sort n items by comparison	$O(n \log n)$	$O(n \log n)$
Keep every old version	$O(1)$ more per update [5]	free

Table 2: Worst-case costs, except for persistence, which is amortised. The purely functional array is a balanced tree, and the persistent structure is linked, with bounded in-degree.

References

[1] A. W. Appel, SSA is functional programming, *ACM SIGPLAN Notices* 33(4), 1998.

[2] R. Bird, G. Jones, and O. de Moor, More haste, less speed: Lazy versus eager evaluation, *Journal of Functional Programming* 7(5), 1997.

[3] A. Church, An unsolvable problem of elementary number theory, *American Journal of Mathematics* 58(2), 1936.

[4] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, Efficiently computing static single assignment form and the control dependence graph, *ACM Transactions on Programming Languages and Systems* 13(4), 1991.

[5] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, Making data structures persistent, *Journal of Computer and System Sciences* 38(1), 1989.

[6] M. Felleisen, On the expressive power of programming languages, *Science of Computer Programming* 17(1–3), 1991.

[7] GCC Project, GCC 4.0 release series: Changes, new features, and fixes, `https://gcc.gnu.org/gcc-4.0/changes.html`, accessed 29 October 2026.

[8] R. Hood and R. Melville, Real-time queue operations in pure LISP, *Information Processing Letters* 13(2), 1981.

[9] P. Hudak, J. Hughes, S. Peyton Jones, and P. Wadler, A history of Haskell: Being lazy with class, *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, 2007.

[10] R. A. Kelsey, A correspondence between continuation passing style and static single assignment form, *ACM SIGPLAN Notices* 30(3), 1995.

[11] S. C. Kleene, λ-definability and recursiveness, *Duke Mathematical Journal* 2(2), 1936.

[12] C. Lattner and V. Adve, LLVM: A compilation framework for lifelong program analysis & transformation, *Proceedings of the International Symposium on Code Generation and Optimization*, 2004.

[13] LLVM Project, LLVM's analysis and transform passes, `https://llvm.org/docs/Passes.html`, accessed 29 October 2026.

[14] S. Marlow and S. Peyton Jones, The Glasgow Haskell Compiler, in A. Brown and G. Wilson (eds.), *The Architecture of Open Source Applications*, Volume II, 2012.

[15] C. Okasaki, Simple and efficient purely functional queues and deques, *Journal of Functional Programming* 5(4), 1995.

[16] S. L. Peyton Jones, Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine, *Journal of Functional Programming* 2(2), 1992.

[17] N. Pippenger, Pure versus impure Lisp, *ACM Transactions on Programming Languages and Systems* 19(2), 1997.

[18] H. Søndergaard and P. Sestoft, Referential transparency, definiteness and unfoldability, *Acta Informatica* 27(6), 1990.

[19] G. L. Steele Jr., Debunking the 'expensive procedure call' myth or, procedure call implementations considered harmful or, LAMBDA: The ultimate GOTO, *Proceedings of the 1977 Annual Conference of the ACM*, 1977.

[20] C. Strachey, Fundamental concepts in programming languages, *Higher-Order and Symbolic Computation* 13(1–2), 2000. Lecture notes from 1967.

[21] W. Swierstra, Data types à la carte, *Journal of Functional Programming* 18(4), 2008.

[22] A. M. Turing, On computable numbers, with an application to the Entscheidungsproblem, *Proceedings of the London Mathematical Society* s2-42(1), 1936.

[23] A. M. Turing, Computability and λ-definability, *Journal of Symbolic Logic* 2(4), 1937.

[24] University of Manchester, How a 70-year-old Baby changed the face of modern computing, https://www.manchester.ac.uk/about/news/how-a-70-year-old-baby-changed-the-face-of-modern-computing/, 21 June 2018.

[25] P. Wadler, The Expression Problem, email to the java-genericity mailing list, 12 November 1998.

[26] C. P. Wadsworth, *Semantics and Pragmatics of the Lambda-Calculus*, DPhil thesis, University of Oxford, 1971.